<Academia de Código>

SUMMARIZER

30/03/23

<Rodrigo Caçorino>

#81_Bootcamp

Main Topics

SPRING MVC

Architecture
Controller
Model
View Resolver

AUTO WIRING

THYMELEAF

Java Template Engine

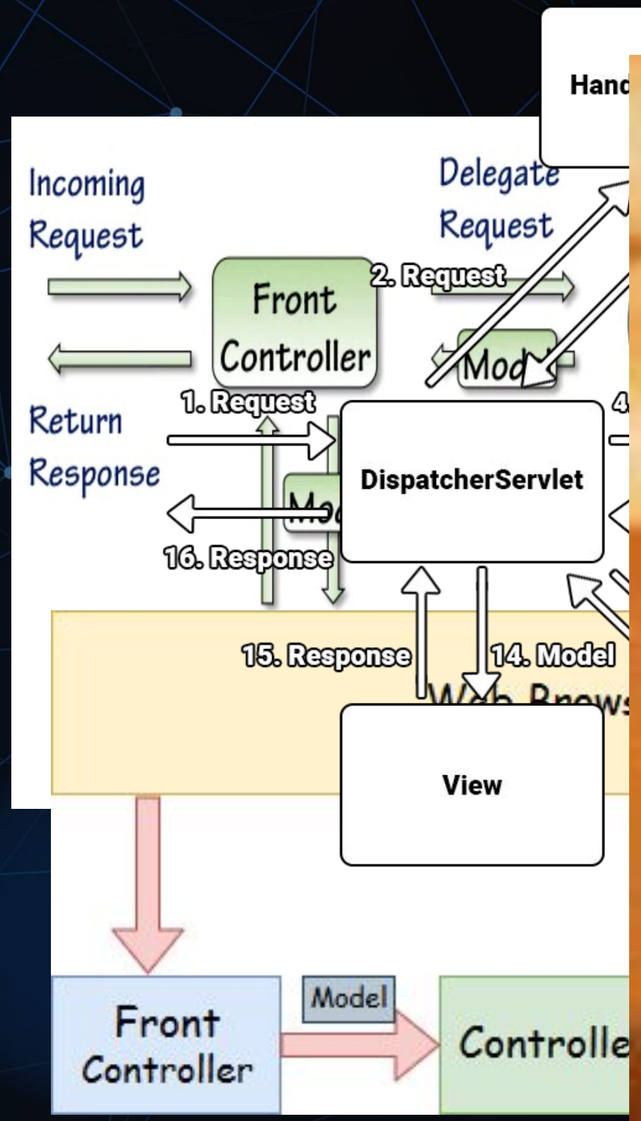
JAVABANK EXERCISE

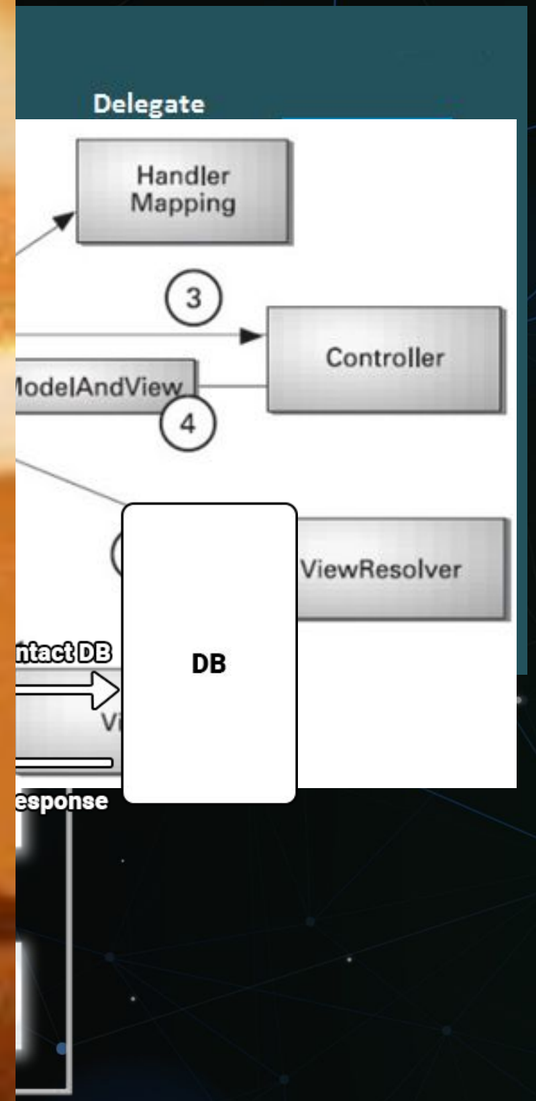
SPRING MVC

- Centered around the Model View Controller design pattern

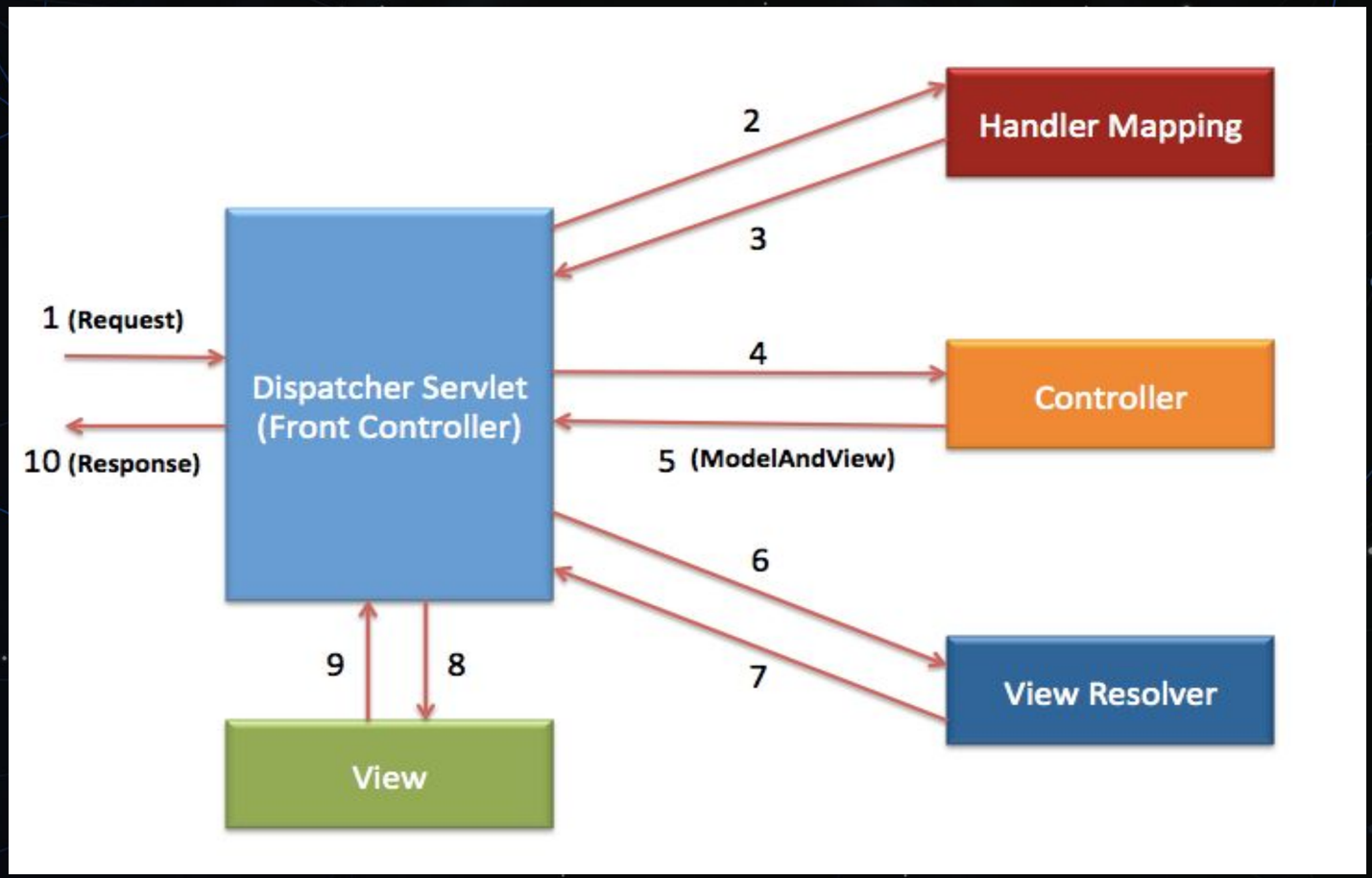
- Implements all the basic features of a core spring framework

Architecture

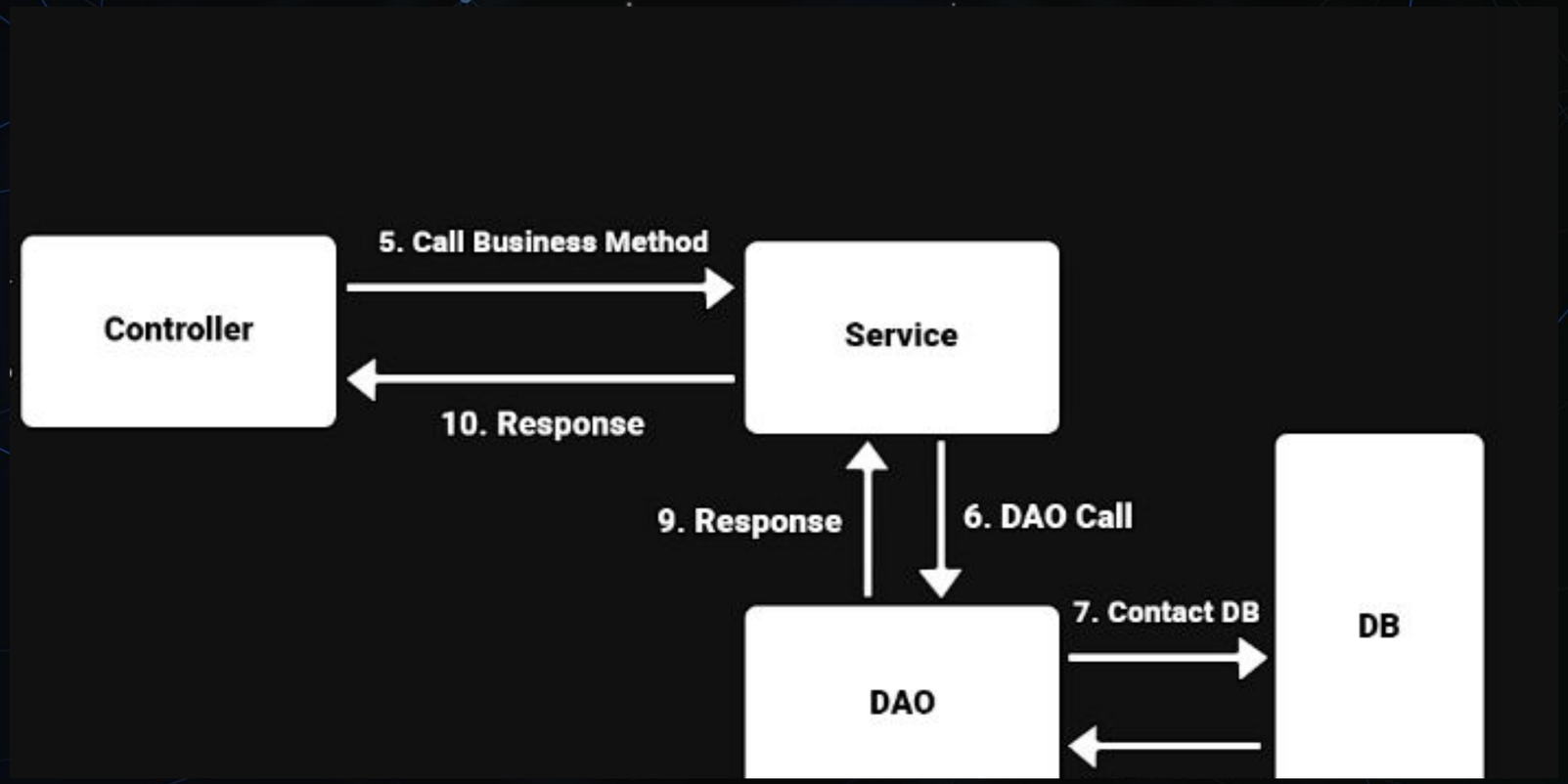




Architecture



Architecture

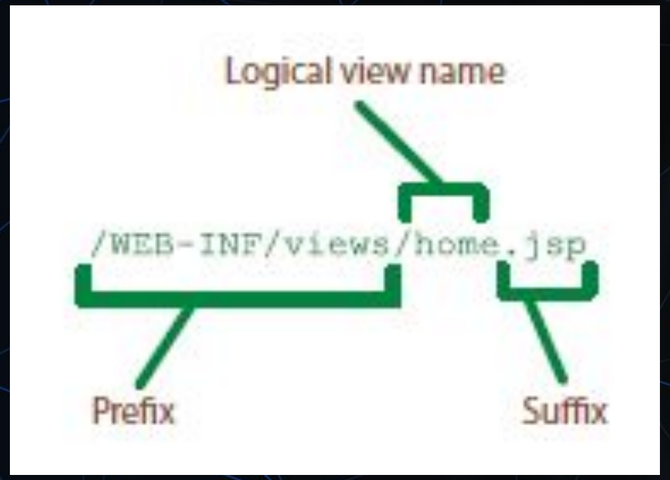


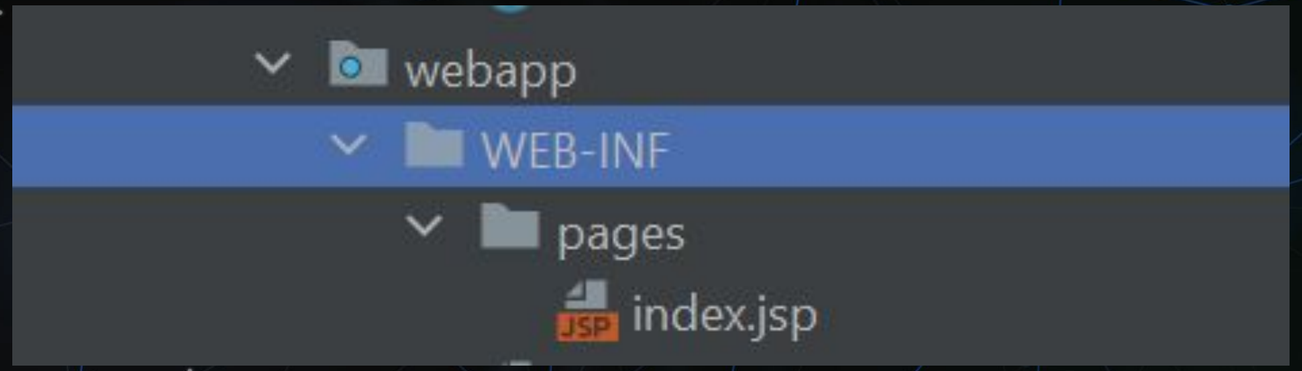
Spring Controller

```java
@Controller
public class newCustomerController {

        Customer customer = new Customer("Rodrigo","Caçorino","916999999","rodrigo@gmail.com");
        // Map the URL/Verb to the method
        @RequestMapping(method = RequestMethod.GET, value = "/bedro")
        public String addStuff(Model model) {

// Add an attribute to the request
            model.addAttribute("first_name", customer.getFirst_name());
            model.addAttribute("last_name", customer.getLast_name());
            model.addAttribute("phone", customer.getPhone());
            model.addAttribute("email", customer.getEmail());

// Return the view name
            return "index";

        }
    }
```

Spring will register controllers and their mappings by scanning your code for the @Controller and @RequestMapping annotations

View Resolver



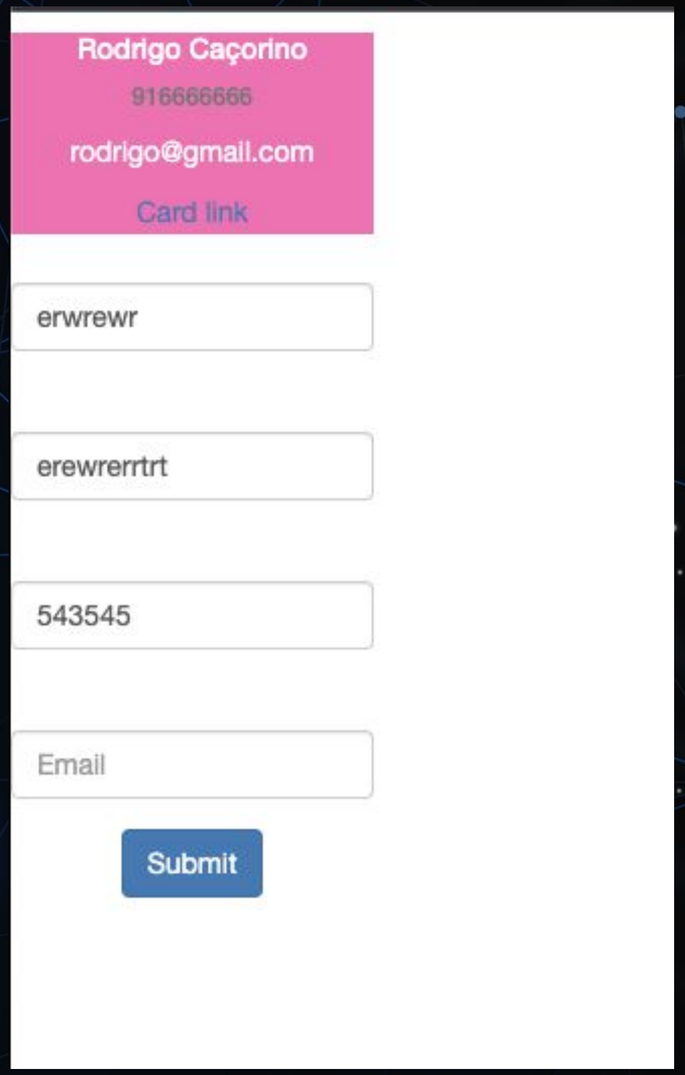



```
<!-- configure the view resolver -->
    <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
        <property name="prefix" value="/WEB-INF/pages"/>
        <property name="suffix" value=".jsp"/>
    </bean>
</beans>
```

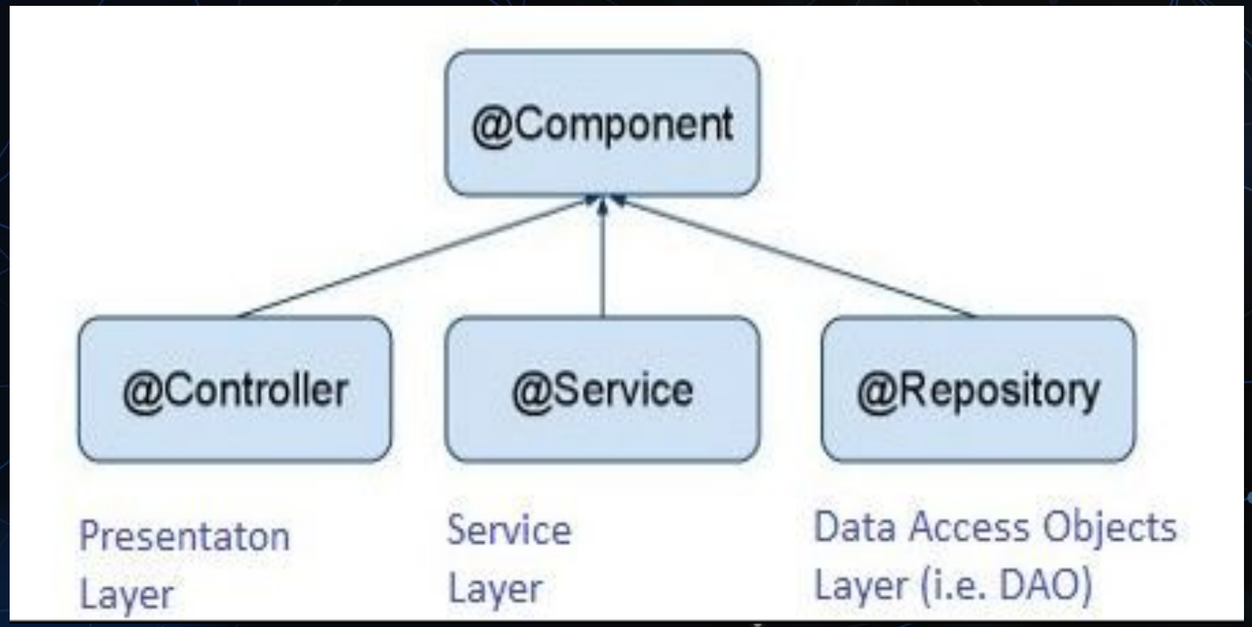
View Template

```jsp
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<html>
<head>
    <title>Customer Card</title>
    <link rel="stylesheet" href="../style.css"/>
    <style><%@include file="/WEB-INF/style.css"%></style>
</head>
<body>

<div class="card" style="width: 18rem;">

    <div class="card-body"  >
        <h5 class="card-title">${first_name} ${last_name}</h5>
        <h6 class="card-subtitle mb-2 text-muted">${phone}</h6>
        <p class="card-text">${email}</p>
    </div>

    <form action="" method="post">
        <div class="form-group">
            <label for="first_name1">First Name ç faz favore</label>
            <input name="first_name" class="form-control" id="first_name1" placeholder="Your First Name">
        </div>
        <div class="form-group">
            <label for="last_name1">Last Name</label>
            <input name="last_name" class="form-control" id="last_name1" placeholder="Your Last Name">
        </div>

         <button type="submit" class="btn btn-primary">Submit</button>
    </form>
</div>

</body>
```

View Template



Auto Wiring



Enables object dependencies to be injected implicitly without the need for explicit configuration

Auto Wiring

```java
@Controller
public class CustomerController {
    private CustomerService customerService;


   @Autowired
    public void setCustomerService(CustomerService customerService) {
        this.customerService = customerService;
    }
  ----------------------------------------
@Service
public class CustomerServiceImpl implements CustomerService {

    private CustomerDao customerDao;
```

```xml
    <context-param>
        <param-name>spring.profiles.active</param-name>
        <param-value>jpa</param-value>
    </context-param>
```

Thymeleaf

Java template engine for processing and creating HTML, XML, JS, CSS and text

Goal is to provide a stylish and well formed way of creating templates

It is a substitute for JSP

Javabank Exercise

JavaBank Customer list view using Spring MVC and Thymeleaf

Javabank Exercise

```java
public interface CustomerService {

    1 usage   2 implementations
    List<Customer> getCustomers();
```

```java
@Service
public class CustomerServiceImpl implements CustomerService {

    6 usages
    private CustomerDao customerDao;
```

```java
1 usage
@Override
public List<Customer> getCustomers() {
    return customerDao.findAll();
}
```

Javabank Exercise

```java
@Controller
public class CustomerController {
    2 usages
    private CustomerService customerService;


    no usages
    @Autowired
    public void setCustomerService(CustomerService customerService) {
        this.customerService = customerService;
    }

    // Map the URL/Verb to the method
    no usages
    @RequestMapping(method = RequestMethod.GET, value = "/")
    public String addStuff(Model model) {

// Add an attribute to the request
        model.addAttribute( s: "customers", customerService.getCustomers());

// Return the view name
        return "customerList";
```

Javabank Exercise

```
<head>
    <title>JavaBank</title>
    <meta http-equiv="Content-Type" content="text/html; charset=utf-8"/>
</head>
<body>
<div>


    <h2>Customer List</h2>
    <table>
        <tr>
            <th>Id</th>
            <th>First Name</th>
            <th>Last Name</th>
            <th>Email</th>
            <th>Phone</th>
        </tr>

        <tr th:each="customer : ${customers}">
            <td th:text="${customer.id}"></td>
            <td th:text="${customer.firstName}"></td>
            <td th:text="${customer.lastName}"></td>
            <td th:text="${customer.email}"></td>
            <td th:text="${customer.phone}"></td>
        </tr>
    </table>
</div>
</body>
</html>
```

Safari File Edit View History Bookmarks Window Help

localhost

Customer List

Id	First Name	Last Name	Email	Phone
1	Rui	Ferrão	mail@gmail.com	777888
2	Sergio	Gouveia	mail@gmail.com	777888
3	Bruno	Ferreira	mail@gmail.com	777888
4	No Accounts	No name	mail@gmail.com	777888

THANK YOU!